Task 1: Data Preparation and Cleaning Report

Data Quality Report

This section details the quality issues identified in the raw datasets (`customers.csv`, `listings.csv`, `orders.csv`) and recommends cleaning strategies for each issue. I examine each dataset in turn and highlight problems such as missing values, inconsistent formats, duplicates, type mismatches, and outliers, along with proposed fixes.

Customers Dataset (customers.csv)

- **Missing Values:** Several customer records have missing fields. For example, some entries have no email address or an undefined city (e.g., one record lists city as `"nan"`). These incomplete records can hinder analysis or join operations. *Recommended Cleaning:* Either remove records with critical information missing or impute placeholders. In this case, I retain these customers but leave missing emails/cities blank or as `NULL` to avoid outright loss of data. Missing data should be handled consistently since many algorithms cannot accept nulls.

- **Inconsistent Date Formats:** The `join_date` field appears in multiple formats (e.g., `"2023-01-15"` vs `"02/20/2023"` vs `"Mar 05, 2023"`). This inconsistency complicates sorting and time-based analysis. *Recommended Cleaning:* Parse all date strings into a standard date format (e.g., ISO `YYYY-MM-DD`). A single date format ensures consistency for downstream processing.

- **Whitespace and Casing Issues:** Some text fields contain leading/trailing spaces or inconsistent casing. For instance, one first name is stored as `" John "` with extra spaces, and some city names have random capitalization or spacing (e.g., `" Kuala Lumpur"` with a leading space). *Recommended Cleaning:* Strip whitespace from names and city fields and standardize casing. I convert city names to a consistent case (e.g., all uppercase) for uniformity. Structural errors like stray spaces and inconsistent capitalization are common data entry issues that should be corrected.

- **Duplicate Entries:** The customers dataset was checked for duplicate customer IDs and found to have none (each `customer_id` is unique). This indicates the customer table is uniquely keyed by `customer_id` as expected. Maintaining unique primary keys is crucial to avoid redundant records and ensure one source of truth for each customer. If any duplicate customer records had existed, I would drop or merge them as needed to prevent skewed analyses.

- **Data Type Anomalies:** All columns in `customers.csv` are stored as text by default, but semantically some have specific types (e.g., dates). The join dates should be date objects rather than strings, and the customer IDs are identifiers that should consistently be strings (the prefix `c` in IDs prevents them from being misinterpreted

as numeric). I did not find numeric outliers in this dataset (there are no numeric measures besides the date); however, ensuring the join_date is a valid date (no future dates beyond a reasonable range) is important. *Recommended Cleaning:* Enforce schema types where possible (e.g., convert `join_date` to date type). This aligns with validity checks in data quality assurance – confirming each field contains the expected type or pattern (dates, emails, etc.). Also, an email regex check must be applied to ensure emails are well-formed.

- **Structural Issues:** Extra "Unnamed" Columns from Misaligned Fields During inspection, each raw CSV (`customers.csv`, `listings.csv`) sometimes produced an extra unnamed column**.** This happened because some rows contained an additional comma, typically inside quoted or malformed text fields such as dates written as `"Mar 05, 2023"` or product names containing commas causing the parser to misinterpret the structure and push part of the row into a new blank column.

Listings Dataset (listings.csv)

- **Inconsistent Price Format and Type:** The `price` field in listings is stored as a string with currency symbols (e.g., `"$29.99"` and in some cases with extra spaces or quotes). This is a type mismatch, as price should be numeric. *Recommended Cleaning:* Remove currency symbols and any stray whitespace, then convert the field to a numeric data type (float). For example, `"$25.00"` would be parsed to `25.00`. This conversion allows mathematical operations on price and ensures the field conforms to a single numeric format.

- **Inconsistent Category Labels:** Product categories show capitalization and spacing inconsistencies. For instance, some entries are `"Electronics"` while another is `" electronics"` (lowercase or with leading space), and `"Fashion"` vs `" Fashion "`. These should represent the same categories but appear as distinct values due to formatting. *Recommended Cleaning:* Normalize the `category` field by stripping whitespace and standardizing case (e.g., use all uppercase categories like `ELECTRONICS`, `FASHION`, etc.). This prevents duplicate categories caused by case sensitivity or typos.

- **Date Format Variability:** The `listing_date` (when the product was listed) also appears in mixed formats (e.g., `"2023-02-01"`, `"03/15/2023"`, and `"Sep 12, 2023"`). *Recommended Cleaning:* Similar to join_date, parse and format all listing dates to a consistent `YYYY-MM-DD` format. Standardizing date formats across datasets is critical for temporal comparisons and ensures the field is stored as a true date type.

- **Stock and Price Range Check:** The `stock_quantity` and cleaned `price` were examined for outliers. No negative quantities or prices were found in the raw data (stock counts are non-negative integers, and prices range within expected values for retail products). The maximum price is under $500 and stock counts are in

reasonable ranges, so no extreme outlier treatment was needed. It is still good practice to validate that these values fall within logical bounds (e.g., no negative stock or zero prices) as part of quality checks. Any values outside expected ranges would be flagged for correction or removal.

- **Duplicates:** Each listing has a unique `listing_id` (e.g., L001, L002, ...). I confirmed there were no duplicate listing IDs in the raw data. This indicates the listings table is clean in terms of primary key uniqueness. If duplicates had been present, they would be removed to avoid one product appearing twice.

- **Structural Issues:** Extra "Unnamed" Columns from Misaligned Fields During inspection, each raw CSV (`customers.csv`, `listings.csv`) sometimes produced an extra unnamed column**.** This happened because some rows contained an additional comma, typically inside quoted or malformed text fields such as dates written as `"Mar 05, 2023"` or product names containing commas causing the parser to misinterpret the structure and push part of the row into a new blank column.

Orders Dataset (orders.csv)

- **Repeated Orders and Duplicate Rows:** The orders file contains multiple rows per `order_id` because each order can have multiple status events (Placed, Shipped, Delivered, etc.). I expect multiple records per order in a **long** format of order events. However, I discovered some exact duplicate rows (identical order_id, status, timestamp, etc.), likely due to data entry or logging errors. *Recommended Cleaning:* Remove true duplicate records to prevent double-counting. De-duplication is important because duplicate observations can skew analysis (e.g., counting an order event twice). After cleaning, each unique event should appear only once.

- **Inconsistent Order Status Values:** There were numerous variants of order status strings (18 unique values) representing a small set of actual statuses. For example, "Placed" vs "placed" vs "PLACED", "Shipped" vs "shiped", "Delivered" vs "delivred", and multiple spellings of "Canceled/Cancelled". These inconsistencies are due to typos and case differences. *Recommended Cleaning:* Normalize the `order_status` field by trimming whitespace and converting all values to a consistent case (e.g., uppercase), then mapping misspellings to the correct standard values. I consolidated all status entries to four canonical values: **PLACED, SHIPPED, DELIVERED,** and **CANCELLED**. For instance, "placedd" and "Placed" both become "PLACED", "delivred" is corrected to "DELIVERED", and all variations of cancel (one *L* or two) are unified to "CANCELLED". This resolves structural errors in categorical data and ensures that analysis by status is accurate.

- **Timestamp Format and Temporal Order:** The `event_timestamp` field in orders appears in a mix of formats (e.g., `"9:33 AM Jul 03, 2023"`, `"2024-01-12 06:01:07"`, `"01/12/2024 07:01:07"`). This inconsistency makes it difficult to sort events chronologically or parse them as datetimes. *Recommended Cleaning:* Parse

all timestamps into a proper datetime data type and standardize the format (e.g., ISO 8601 "YYYY-MM-DD HH:MM:SS"). I used Python's date parsing to interpret each string (handling 12-hour AM/PM formats and different date layouts) and reformatted to a uniform timestamp. After cleaning, `event_timestamp` is stored as a consistent datetime, allowing correct chronological sorting.

- **Timing:** I also cross-checked event timestamps against listing dates and customer join dates for sanity. In a few cases, an order event occurred slightly before the product's listing date (suggesting a data entry timing issue) – these are rare and were not automatically altered, but they highlight the importance of range and cross-field checks (e.g., an order date should not precede the item's listing date). Such records would ideally be flagged for further investigation.

task1_scripts > test.py > ...

```python
import pandas as pd

# Load cleaned datasets
customers = pd.read_csv("clean_customers.csv", parse_dates=['join_date'])
listings  = pd.read_csv("clean_listings.csv", parse_dates=['listing_date'])
orders    = pd.read_csv("clean_orders.csv", parse_dates=['event_timestamp'])

# Merge metadata into orders for validation
orders_check = (orders
    .merge(listings[['listing_id', 'listing_date']], on='listing_id', how='left')
    .merge(customers[['customer_id', 'join_date']], on='customer_id', how='left')
)

# Check for logical inconsistencies
invalid_listing = orders_check[orders_check['event_timestamp'] < orders_check['listing_date']]
invalid_join    = orders_check[orders_check['event_timestamp'] < orders_check['join_date']]

print(f"Orders before product listing: {len(invalid_listing)} rows")
print(f"Orders before customer join:   {len(invalid_join)} rows")
```

bash task1_scripts

```
(.venv) drc@airconwifi-mba ~/proj/whynot/task1_scripts$ python test.py
Orders before product listing: 3212 rows
Orders before customer join:   4268 rows
(.venv) drc@airconwifi-mba ~/proj/whynot/task1_scripts$
```

#test.py is not attached in the .zip file

- **Referential Integrity:** I verified that each `customer_id` in orders exists in the customers dataset and each `listing_id` in orders exists in the listings dataset. This check confirmed there are no orphan references (every order's customer and product can be found in the respective dimension tables). Ensuring that foreign keys point to valid entries is essential for referential integrity. (If any order had an invalid customer_id or listing_id, our cleaning strategy would be to either fix the ID if it's a known typo or remove those records as inconsistent.) Maintaining this referential integrity guarantees that the Orders data can reliably be joined with Customers and Listings without mismatch issues.

- **Outliers and Logical Inconsistencies:** I checked for any out-of-range values or logical anomalies in orders. Quantities are small integers (1–3) as expected (no zero or negative quantities). Prices at purchase range from about $10 to $500, consistent with the product prices. One complex issue observed was that some orders had multiple lines for different listings (e.g., one order ID referring to two different products, each with its own status sequence). This indicates multi-item orders, which is acceptable, but it required careful handling: all events for a given order+item combination were kept separate.

Cleaning Pipeline Summary

The data cleaning was implemented in a Python script (`clean_data.py`) to ensure the process is reproducible and auditable. By using a scripted pipeline, I ensure that every run applies the same transformations in the same order, which is critical for reproducibility and consistency across environments. Below is a summary of the key transformations and how major issues from the Data Quality Report were addressed:

- **Reading and Type Inference:** The raw CSV files were read using pandas. No schema was enforced on read, so initial data types were inferred (e.g., IDs as strings, numeric fields as floats/ints where possible, dates as strings). The cleaning steps then cast or converted types explicitly where needed (e.g., parsing dates, converting prices to numeric).

- **Removing Duplicates:** Duplicate records in the orders dataset were dropped to eliminate redundant events. This was done using a command like `orders.drop_duplicates(inplace=True)`, which removes any completely identical rows. By enforcing primary key uniqueness and removing duplicate entries, I uphold data integrity (no repeated records violating uniqueness constraints). For example:

```
# Remove exact duplicate rows in orders data
orders = orders.drop_duplicates()
```

- **Trimming and Standardizing Text Fields:** All text fields such as names, cities, and categories were stripped of leading/trailing whitespace and normalized in case. For instance, customer names and cities were stripped and uppercased, and product categories were trimmed and uppercased for consistency:

```
# Strip whitespace and normalize case
customers['first_name'] = customers['first_name'].str.strip()
customers['last_name']  = customers['last_name'].str.strip()
customers['city']       = customers['city'].str.strip().str.upper()
listings['category']    = listings['category'].str.strip().str.upper()
```

This step fixes the inconsistent capitalization and spacing issues (e.g., `" Fashion "` becomes `"FASHION"`). As a result, categories and city names can be grouped or compared without spurious differences.

- **Standardizing Date/Time Formats:** A critical part of the pipeline was parsing date and time strings into a uniform format. I leveraged pandas datetime parsing to interpret various formats and then formatted the dates consistently:

```
# Standardize date fields
customers['join_date']    = pd.to_datetime(customers['join_date'],
infer_datetime_format=True).dt.strftime('%Y-%m-%d')
listings['listing_date']  = pd.to_datetime(listings['listing_date'],
infer_datetime_format=True).dt.strftime('%Y-%m-%d')
orders['event_timestamp'] = pd.to_datetime(orders['event_timestamp'],
infer_datetime_format=True)
```

In this snippet, `infer_datetime_format=True` allows pandas to parse each date string whether it's in ISO, slash format, or text month format. I then format `join_date` and `listing_date` as `YYYY-MM-DD` strings. For `event_timestamp`, I parsed it to a full datetime (the output was stored in a datetime format, and when writing to CSV, it appears as a standardized timestamp string). This conversion resolves the mixed date formats issue, allowing chronological sorting and time interval calculations. Any parsing failures (e.g., truly malformed dates) would result in `NaT` (Not-a-Time) values, which I did not encounter in our dataset after using the infer mechanism.

- **Cleaning Numeric Fields:** The pipeline cleaned the price field in listings by removing currency symbols and converting to float:

```
# Remove '$' and convert price to numeric
listings['price'] = listings['price'].str.replace(r'[$,]', '',
regex=True).astype(float)
```

This transformation removed characters like `$` and any commas (for thousands, if present) from the price, then cast the result to a float. The `price` column in `listings.csv` thus became a numeric field (e.g., `"$129.00"` → `129.00`), enabling arithmetic operations. I similarly ensured `stock_quantity` was an integer type. After this step, the listings data has proper numeric types for price and stock.

- **Order Status Normalization:** To tackle the variety of `order_status` values, I applied string normalization and mapping:

```
# Normalize order status values
orders['order_status'] = orders['order_status'].str.strip().str.upper()
orders['order_status'] = orders['order_status'].replace({
    'CANCELED':  'CANCELLED',   # unify spelling
    'CANCELEDD': 'CANCELLED',
    'DELIVRED':  'DELIVERED',
    'PLACEDD':   'PLACED',
    'SHIPED':    'SHIPPED'
})
```

First, I strip whitespace and convert everything to upper case (so "Delivered", "delivered", "DELIVERED" all become "DELIVERED"). Then I replace known misspellings with the correct terms. After this mapping, I confirmed that `order_status` has only the four

expected values. This standardization is crucial because, for example, without it an analysis might treat "Shipped" and "shipped" as different categories when they are semantically the same. By correcting these values, I eliminate structural errors and typos in the status field.

- **Ensuring Referential Integrity:** During cleaning, I did a cross-check to ensure consistency between datasets. Using inner merges or set comparisons, I verified that every `customer_id` in orders exists in the cleaned customers table and every `listing_id` in orders exists in the cleaned listings. The cleaning script could log or assert that no foreign-key mismatches are present. In our case, the data was already referentially intact, so no further action was needed beyond the check. This kind of **consistency check** (verifying that every order has a valid customer and listing) is a best practice in data engineering to maintain schema integrity. If a mismatch had been found, the strategy might be to remove those orphaned order records or to correct the foreign keys if the intended match is known.

- **Final Schema and Output:** After cleaning, the three datasets were written out as `clean_customers.csv`, `clean_listings.csv`, and `clean_orders.csv`. The cleaned data is now structured and typed as follows:

- **Customers:** 6 columns (`customer_id`, `first_name`, `last_name`, `email`, `join_date`, `city`). All text fields are trimmed; `join_date` is standardized (e.g., `2023-03-05`); `city` is uppercase. Missing emails or cities remain blank/NULL (to be handled in analysis as needed). The table retains one row per customer, maintaining a unique customer_id for each (ensuring each customer is only represented once).
- **Listings:** 7 columns (`listing_id`, `product_name`, `category`, `seller_id`, `price`, `stock_quantity`, `listing_date`). The `price` is numeric (float) without currency symbols, `stock_quantity` is integer, and `listing_date` is uniform `YYYY-MM-DD`. Categories are all uppercase and free of extra spaces. Each listing_id is unique, representing one product listing.
- **Orders:** 7 columns (`order_id`, `customer_id`, `listing_id`, `quantity`, `price_at_purchase`, `order_status`, `event_timestamp`). The `event_timestamp` is in a consistent datetime format, and `order_status` is one of {PLACED, SHIPPED, DELIVERED, CANCELLED} only. The data is in long format with multiple entries per `order_id` (each representing a status change event for a particular item in the order). Within each order, the events can be chronologically ordered by the timestamp. All `customer_id` and `listing_id` values here should correspond to entries in the cleaned customers and listings tables (referential integrity ensured). Primary keys are composite in the sense that an `order_id` can repeat (not unique by itself), but the combination (order_id, listing_id, order_status, timestamp) is unique per event. I also ensured no exact duplicate events remain.

Throughout the pipeline, I followed data engineering best practices such as keeping the raw and clean data separate, using code for transformations (which documents the steps clearly), and maintaining the normalized structure of the database. The separation of

customers, listings, and orders into three related tables is an example of a normalized schema design that reduces redundancy and preserves referential integrity. For instance, any given product price or customer detail is stored in one place (listings or customers table), and the orders table references them via IDs. This makes the data model consistent and updateable without anomalies – a new customer or product can be added independently, and orders link to them via foreign keys.

Finally, the cleaning pipeline is reproducible: if new raw data arrives or if I need to re-run the cleaning, the same code will produce the same structured output. This ensures that the process is not ad-hoc but repeatable and version-controlled. Reproducible pipelines are crucial for trust in data, as they guarantee that the cleaning steps can be applied consistently across different data refreshes or environments, supporting the overall data integrity over time.

Task 2: Relational Data Modeling and Loading (MySQL with Docker)

Normalized Schema Design (3NF)

The relational schema is composed of five tables – Customers, Listings, Orders, Order_Items, and Order_Item_Status_History – organized in at least Third Normal Form (3NF). Each table represents a single entity or relationship, eliminating duplicate data and ensuring that non-key attributes depend only on the primary key. This normalization reduces redundancy and enforces integrity by "splitting complex tables into smaller, logically structured ones". In 3NF, there are no transitive dependencies; for example, an Order_Item_Status_History record's status text depends on the composite key (order ID, listing ID, timestamp) and not on any other non-key field, satisfying 3NF. By organizing data into discrete tables with well-defined keys, the schema avoids update anomalies and inconsistencies that plague denormalized designs.

The schema uses foreign keys to maintain referential integrity between tables. For example, Orders.customer_id is a foreign key referencing Customers.customer_id, so every order record must refer to a valid customer. Similarly, Order_Items has foreign keys to Orders and Listings, and Order_Item_Status_History has a composite foreign key referencing Order_Items. These constraints ensure data integrity: one cannot insert an order with a non-existent customer, or a status record without a corresponding order item. The foreign key rules enforce "child" records referencing a valid "parent" record, preventing orphans and inconsistent relationships. Where appropriate, the schema uses ON DELETE actions to handle cascades: deleting an Order will CASCADE delete its Order_Items (and those in turn will cascade-delete related history records), whereas deleting a Listing is RESTRICTED if it's referenced by any Order_Item (to preserve order history).

Data Modeling Trade-offs and Design Decisions

In designing this schema, several modeling trade-offs were carefully considered to balance normalization, simplicity, and performance:

- **Inline vs. Reference Tables (Denormalization):** By requirement, certain attributes like `category` and `seller_id` are stored *inline* in the **Listings** table, rather than as separate lookup tables. This denormalization introduces some redundancy – e.g. the category "ELECTRONICS" or seller name "TechGuruTom" appears in multiple listing rows – but simplifies queries by avoiding extra joins. The trade-off is that data consistency for those fields relies on convention rather than foreign key constraints. In a fully normalized design, I might have separate **Categories** and **Sellers** tables with their own keys, ensuring each category or seller name is stored once. However, the chosen approach prioritizes simplicity and reflects a conscious denormalization to "simplify access" at the **cost of some redundancy**. Since categories and sellers

in our dataset form a small, relatively static set of values, the risk of anomalies (e.g. renaming a category) is low and manageable. This decision illustrates the general normalization vs. denormalization trade-off: normalization reduces duplication and anomalies, while denormalization can boost read performance or simplicity often at the cost of consistency. Here I accepted minimal redundancy (repeating category/seller text) in exchange for a more straightforward schema as required.

- **Composite Primary Keys and Indexes:** The use of composite keys (in Order_Items and Order_Item_Status_History) reflects a design choice to capture real-world multi-attribute identifiers. Each composite PK comes with supporting indexes to optimize access patterns. For example, the primary key on Order_Items (`order_id, listing_id`) inherently serves as an index on order_id (and listing_id as second part). This speeds up finding all items of a given order (matching the leading `order_id`). To also support queries by listing (e.g. "find all orders containing listing L050"), a separate secondary index on `listing_id` is provided. Similarly, Order_Item_Status_History's PK (`order_id, listing_id, status_timestamp`) is indexed on that composite; additional indexes on `order_status` and `status_timestamp` alone were created for potential analytic queries (e.g. retrieving all occurrences of a particular status, or sporting events by time)

Mapping the Schema to Source Data (CSV Integration)

The normalized schema was derived to fit the structure of the provided clean CSV datasets. I ensured that every data element in the source files maps to a field in the schema, and conversely that the schema can represent all the source data without loss. The three input files – `clean_customers.csv`, `clean_listings.csv`, and `clean_orders.csv` – correspond to the main entity tables and were used to populate them.

- **Orders and Order Details:** The `clean_orders.csv` is the most complex source file, as it actually contains order item records with status events, rather than a single row per order. Each line of this CSV represents a status update for a particular item in an order, with columns: *order_id, customer_id, listing_id, quantity, price_at_purchase, order_status, event_timestamp*. For example, consider Order 5004 by customer c064 in the CSV: this order appears across multiple lines:

- *Order 5004, customer c064, listing L019, quantity 1, price 12.55, status PLACED at 2024-01-28 10:51:00*

- *Order 5004, customer c064, listing L019, quantity 1, price 12.55, status SHIPPED at 2024-01-29 04:51:46*

These two lines represent the same order (5004) and same item (listing L019) progressing through two status events (PLACED and SHIPPED). In the source, the `quantity` and `price_at_purchase` are repeated on each status line for clarity. Similarly, order 5004 has additional lines for other listings in that order (L028, L029, L088) and their statuses. To fit this data into our schema, I perform a decomposition: each unique order and each unique

order-item combination are extracted, and status events are separated into their own table:

- **Orders table:** I create one Orders record for each unique `order_id`. In the CSV, order 5004 appears on multiple lines, but *all* those lines share the same `order_id=5004` and `customer_id=c064`. I take one representative (or simply deduplicate by order_id) to populate Orders (5004, c064). Thus, Orders has a single row per order, capturing the customer relationship. For order 5004, the Orders table row is `(order_id=5004, customer_id='c064')`. This satisfies the foreign key to Customers: customer c064 is present in the customers table (e.g., c064 is "Brenda Richardson"). Across the whole dataset, there are 800 unique orders, each now represented once in Orders (with the correct customer). The referential integrity between Orders and Customers is maintained by the loading process: I only insert orders after all customers are inserted, and every `customer_id` in orders was verified to exist in the customers dataset (the foreign key ensures this at insertion time).

- **Order_Items table:** For each unique combination of (order_id, listing_id) in the orders CSV, I create an Order_Items row capturing that item's `quantity` and `price_at_purchase`. In the source, an order-item may appear on multiple lines (one per status), but those lines have identical quantity and price. I thus take the data from the *first* (or any) occurrence to avoid duplication. For example, order 5004 has listing L019 with quantity 1 and price 12.55 (from the CSV), listing L028 with quantity 2 price 74.71, L029 with quantity 3 price 367.54, and L088 with quantity 1 price 357.50. These yield four Order_Items records: `(5004, L019, qty=1, price=12.55)`, `(5004, L028, qty=2, price=74.71)`, `(5004, L029, qty=3, price=367.54)`, `(5004, L088, qty=1, price=357.50)`. Each of these rows corresponds to an entry that was repeated in the CSV but now is stored once. This **eliminates the redundancy** of having the same price/quantity copied on multiple status lines – a direct benefit of normalization. In fact, for order 5004 the source CSV held 9 lines (status events) containing the price 367.54 three times (for listing L029) and so on; in our schema that price is stored only once in Order_Items, reducing duplication and potential inconsistency.

- **Order_Item_Status_History table:** Every line of the orders CSV becomes one record in Order_Item_Status_History, capturing the `order_status` and `timestamp` for the given order-item. In the case of order 5004, this yields 9 history records, for example: `(5004, L019, "2024-01-28 10:51:00", "PLACED")`, `(5004, L019, "2024-01-29 04:51:46", "SHIPPED")`, `(5004, L029, "2024-01-28 12:39:00", "CANCELLED")`, `(5004, L088, "2024-01-31 10:52:55", "DELIVERED")`, etc. These are inserted into the Order_Item_Status_History table with the same composite key as in Order_Items plus the timestamp. The schema's composite foreign key `(order_id, listing_id)` on this table ensures that each status record corresponds to an existing Order_Item. In our loading process, I generate these history rows after populating Order_Items, so referential integrity is intact – e.g. I

only insert a status (`5004, L029, …`) after confirming (`5004, L029`) exists in Order_Items. The history data preserves the exact status text from the CSV (e.g. "SHIPPED", "DELIVERED") without normalization of the status values – statuses are treated as free text here. That is acceptable given the context (a finite set of known status strings), though in a larger system one might use an ENUM or separate status reference table.

Data Engineering Considerations: Integrity, Reproducibility, and Efficiency

- **Data Integrity and Consistency:** The schema's constraints (primary keys, foreign keys, not-nulls) act as built-in data quality checks, as discussed. **Referential integrity** is guaranteed by the use of foreign keys – the database itself will reject any operation that violates parent-child consistency. For instance, if the Orders CSV had referenced a customer "c999" not in Customers, the insert would fail – thus I *trust but verify* the source data via constraints. In our case, each foreign key column is indexed (either as part of a PK or via an explicit index like `idx_orders_customer` on Orders), so referential checks are fast. I also enforce entity integrity: every table has a primary key (no duplicate rows), and the primary keys themselves (like customer_id or listing_id) were verified to be unique in the source ("PK uniqueness" is inherently satisfied by the CSV prep step and the PRIMARY KEY constraints in SQL). The database was set to **SQL STRICT mode**, meaning any violation of data types or lengths results in an error rather than silent truncation.

- **Reproducibility and Automation:** The entire schema creation and loading process is scripted and containerized (using MySQL in Docker), which enhances reproducibility. The schema DDL (`schema.sql`) can be run to recreate the database structure exactly, and the loading script (`load_to_mysql.py`) handles reading the CSVs and inserting the data in the correct order. Because this process is code-driven, it can be re-run at any time (for example, if I receive updated CSVs or need to deploy to a new environment) and I will get the same result – a key aspect of reproducible data engineering pipelines. The loading script is designed to be idempotent and resilient.

- **Loading Order and Referential Dependencies:** A critical implementation detail was loading data in an order that respects table dependencies. I explicitly load Customers first, then Listings, then Orders, then Order_Items, then Status_History. This ordering guarantees that when I insert into a child table, its parent records already exist.

The `transform_orders` function in the script encapsulates the logic described in the previous section: it drops duplicates to get one row per order (for Orders table), and one row per order-item (for Order_Items) – choosing the latest event for each order-item to pick up final quantity/price (in our data, quantity/price don't change over time, so first or last event yields the same values). This approach ensures that if an item was canceled, I still

capture its quantity and price (since those remain on the last status line, e.g. “CANCELLED”) and don’t accidentally omit it.

- **Performance and Efficiency:** The volume of data here is modest (a few hundred to a few thousand rows per table), but I still applied good practices to ensure loading is efficient. Using batch inserts (in sets of 1000) amortizes the overhead of SQL insert operations and significantly speeds up the load versus inserting each row individually. Even with foreign key checks active, the batch approach and proper indexing keep insert performance acceptable.

- **Data Integrity Validation:** After loading, I performed row-count and referential checks as a final validation. The number of rows inserted in each table matched expectations from the CSVs (e.g. 200 customers, 100 listings, 800 orders, 2446 order_items, 6770 history events), confirming no data was lost or duplicated. The foreign key constraints implicitly validate referential integrity – since the transaction committed successfully, I know every Order’s customer existed, every Order_Item’s order and listing existed, etc. This is essentially the “load validation” step of the pipeline, where I ensure all records were loaded and all references resolve. I also spot-checked a few join queries (like the example with order 5004 above) to verify correctness.

Analytical Querying for SocialShoppingSG (Task 3)

Stakeholder-Specific SQL Insights

Marketing/Sales – Top Influencers by Category Revenue

Objective: Identify the top **X** influencers (sellers) ranked by total sales revenue *within each product category*, considering only orders that were delivered (completed). This helps Marketing focus on high-performing sellers in each segment.

SQL Approach: The query must aggregate revenue by seller and category, then rank sellers per category. Key steps include:

- **Filter delivered orders:** Use a `WHERE` clause to include only orders with a delivered status (e.g. `o.status = 'DELIVERED'`). This ensures revenue is based solely on completed sales.
- **Join relevant tables:** Combine the orders, order_items, and products (or categories) tables to associate each order item with its product category and seller. Typically, `OrderItems` links to `Orders` (via `order_id`) and `Products` (via `product_id`), and each `Product` has a seller (influencer) and category attribute.
- **Aggregate revenue:** Sum the item revenue per seller *per category*.
- **Rank sellers within each category:** Apply a window function `RANK()` partitioned by category, ordering by total_revenue descending.
- **Retrieve the top X per category:** Wrap the above in an outer query (or use a `WITH ranked AS (...)` CTE) and filter to `rank_in_category <= X`. This yields only the top *X* influencers for each product category.

Logistics – Average Time from Order Placement to Shipping

Objective: Calculate the average fulfillment lead time, i.e. the average duration between when an order is **PLACED** and when it is **SHIPPED**. This metric (often measured in days) is critical for Logistics to identify delays in processing orders.

SQL Approach: I compute the time difference between the `PLACED` timestamp and the `SHIPPED` timestamp for each order (or each order item) and then average those differences.

Management – Month-over-Month Revenue Growth

Objective: Compute the **month-over-month (MoM) growth** in revenue for delivered orders. Management is interested in how revenue is trending each month – for example, to see if sales are growing by 5% every month or if there are seasonal dips.

SQL Approach: I need to calculate total delivered-order revenue for each month, then compare each month's revenue to the previous month. The solution uses a subquery or CTE to get monthly revenues and a window function (`LAG`) to obtain the prior month's value for each month's row.

Performance Bottlenecks and Indexing Strategies

Analytical queries can be heavy, scanning large portions of data. I identify potential performance bottlenecks in the above queries and propose indexing strategies to mitigate them:

- **Full Table Scans on Filtered Columns:** Queries that filter by `status='DELIVERED'` (or any column) can trigger a full table scan if no suitable index exists. For instance, selecting only delivered orders from an Orders table with millions of rows would require reading every row (`WHERE status='DELIVERED'`), as the database must check each row's status. This is inefficient if delivered orders are a subset. **Indexing the status column** is a clear optimization: a b-tree index on `Orders(status)` lets the database jump directly to "DELIVERED" entries, avoiding scanning irrelevant ones.

- **Joins on Foreign Keys:** Each join in our queries (orders → items, items → products, etc.) can become a bottleneck if the join keys are not indexed. For example, joining OrderItems to Orders on `order_id` is fast only if one side (typically OrderItems) has an index on `order_id`. Most relational databases do *not* automatically index foreign key columns, so it's imperative to create indexes on columns like `OrderItems.order_id` and `OrderItems.product_id`. Properly indexing foreign keys *greatly accelerates* join performance and prevents full scans of the child table for each row of the parent.

Natural Language to SQL script

#disclaimer this was just a fun idea and I though it was worth exploring. However this is not set up to work on another machine yet and many variables are hardcoded locally. Do give it a shot if you happen to be a brave soul, but hopefully not too many shots – OpenAI API calls are too expensive ☹

Design of a Denormalized Order Status Document for Fast Read Access

Introduction

In a social shopping application, the "Order Status" page must quickly display all relevant details of an order (customer info, items, status timeline, etc.) in a single view. To optimize for fast reads, I adopt a **denormalized NoSQL data model** using MongoDB, where each order is stored as one JSON document containing everything needed by the order status page. This design avoids expensive joins or multiple queries at runtime by keeping together data that would otherwise reside in separate relational tables. The following report presents the proposed document structure, explains the rationale for embedding (vs. referencing) in this read-heavy use case, contrasts it with a normalized SQL schema, and reflects on the trade-offs and scalability of the approach.

Document Structure Design

Each **order** is represented as a single MongoDB document, structured to embed all information required for the order status view.

Embedded data fields: In this design, I **embed product information** within each order item (e.g. product `name`, category, and `price_at_purchase` are stored in the item entry). I also embed a **customer sub-document** with key details like name and shipping address, instead of just a foreign key. Each order's `status_timeline` is stored as an array of status records (with status and timestamp), providing a full history of status changes.

- **Order Metadata:** Order ID (used as `_id` for fast lookup), timestamps for key events (order placed/shipped/delivered), and the current status.
- **Customer Details:** Embedded object with the customer's ID and contact info (e.g. name, email) and shipping address. This avoids an extra query to a customers collection when rendering the order.
- **Items Array:** A list of item sub-documents, each including the `listing_id` and quantity, along with an embedded **product snapshot** (product name, category, seller, and the price at the time of purchase). Storing `price_at_purchase` and computing `extended_price` (price × quantity) at order time ensures the historical pricing is preserved and readily displayed.
- **Payment Info:** Payment method and monetary amounts (subtotal, any shipping or tax, total). By storing the total cost explicitly, the application can display the order summary without recalculating it on the fly.
- **Status History:** An array `status_timeline` tracking each status change (e.g. *PLACED*, *SHIPPED*, *DELIVERED*) with timestamps, providing an audit trail of the order's lifecycle. The latest status is also kept as a top-level `current_status` for convenience.

This single-document model is intentionally denormalized (data is duplicated across orders and embedded sub-documents) to optimize for fast reads.

Justification and Trade-offs

The denormalized document design is chosen because it aligns with the access patterns of the Order Status page, which is a read-heavy use case. In MongoDB schema design, a guiding principle is that "things that are queried together should stay together". By embedding related data (customer info, items, status records) into a single order document, I ensure that all data needed to render the page is retrieved in one database operation, minimizing latency and avoiding complex join logic.

Why embedding is preferable: In our scenario, once an order is placed, the core details (items purchased, prices, customer info) remain mostly static. Embedding a product snapshot in each order is sensible because product attributes like name or price *at purchase time* will not change for that historical order. By copying these fields into the order, I preserve a consistent snapshot of the purchase. This avoids having to join with a products collection to fetch names or prices and ensures that even if the product's data changes later (e.g. the product is renamed or repriced), the order record remains an immutable record of the transaction. The read-to-write ratio here is high – an order might be written once and updated a few times (status updates) but could be read many times by the customer or customer service.

Trade-offs and considerations: The primary trade-off of this denormalized approach is data duplication and potential update complexity. Information like customer name or product details will be stored within each order, rather than referenced via foreign keys. This duplication means a higher storage footprint and the possibility of inconsistencies if the same data needs to change.

Another consideration is the **document size limit** in MongoDB. Each document cannot exceed 16 MB in size. Our design must ensure that even orders with many items will stay within this limit. In practice, an order would rarely approach this size – product fields are small, and even 100+ items (an unusually large order) would typically still be well under 16 MB. For instance, a text field like a product name of ~50 bytes across 100 items is only a few kilobytes. As noted in a MongoDB discussion, 16 MB is quite large and a products array of even "100+ products" is unlikely to hit this limit. If an order ever did approach the document size limit (e.g. an extreme case with thousands of items), I might need to reconsider the model (such as sharding the order or storing items in a separate collection). But given the expected use (normal consumer orders with tens of items at most), the embedded model is safe and far more efficient for reads in the common case.

Write amplification is another trade-off: denormalization can increase the cost of certain writes. For example, inserting a new order document in this model means writing not just an order ID and references, but the full item details and customer data. This one-time cost is higher than in a normalized schema (which would insert smaller records into multiple tables). Similarly, updating a field that is duplicated (if ever necessary) would require

updating it in all orders that have that field. However, these costs are justified by the dramatically reduced read complexity.

Critical Reflection on Assumptions and Scalability

In designing this data model, I assumed that fast read performance for the Order Status page is paramount, and that the pattern of access is read-intensive relative to writes. This assumption holds true for many e-commerce or social shopping scenarios: once an order is placed, a user might check its status multiple times (especially if waiting for delivery), whereas the order data changes only occasionally (status updates, or perhaps cancellations). This high read-to-write ratio underpins the decision to denormalize.

One must consider transactional vs analytical use for this schema. The order documents as designed form an *operational* dataset optimized for serving the live application (transactional reads of single orders). For transactional consistency, operations on a single order (like updating its status) are atomic at the document level, which is sufficient for our needs. I avoid multi-document transactions by keeping an entire order's state in one document. This is well-suited to the transactional workload of user-facing queries ("show me my order status").

Finally, it is important to reflect on whether this model fits a transactional consistency need or is more for read throughput. Since our use case is read-heavy and each order is mostly independent, I do not require multi-document transactions or strict global consistency beyond the single order. Each order document can be updated independently (e.g. when a status change occurs, I update that one document). This localizes any transactional concerns (and MongoDB's single-document ACID properties suffice). For cross-order operations (say, applying a status update to many orders at once), our design doesn't hinder that either – it would just be multiple write operations, which could be managed in an application-level transaction if needed.

Task 5: Documentation of Use of AI

Overview of AI Usage

Throughout this assignment, I used ChatGPT (GPT-5) as a *collaborative assistant*—not as a code generator to replace reasoning, but as a technical and reflective partner to accelerate design, documentation, and debugging across Tasks 1–4. My engagement was iterative: I provided raw data samples, partial code, or conceptual descriptions, and prompted ChatGPT to produce reports, scripts, or schema fragments which I then refined and verified manually.

I used AI for the following key stages:

- Task 1 (Data Preparation and Cleaning): brainstorming systematic data-quality checks, drafting a Pandas-based cleaning pipeline (clean_data.py), and writing the *Data Quality Report*.
- Task 2 (Relational Data Modeling & Loading): refining MySQL schema design for 3NF compliance, clarifying primary/foreign key relationships, and generating explanatory text for the report.
- Task 3 (Analytical Querying): outlining SQL queries for business questions (e.g., top influencers by revenue, logistics timing analysis) and identifying potential query performance bottlenecks.
- Task 4 (NoSQL Data Modeling): structuring a comparative discussion between relational and document-based designs, and drafting placeholders for MongoDB query outputs.

The typical prompting workflow involved short, context-rich instructions such as:

"Generate a concise 2–3 page report for Task 2 focusing on relational modeling trade-offs, referencing 3NF, key constraints, and Docker reproducibility."
"Give me evidence of inconsistent date formats and how I could show that in the report."
"Explain how to justify a schema choice critically"

Example of High-Quality Output

A particularly effective interaction occurred during Task 2, when I asked ChatGPT to critique and explain my SQL schema (schema.sql) and *load script* (load_to_mysql.py). The model generated a clear, design-rationale narrative covering:

- normalization to 3NF,
- enforcement of referential integrity,
- indexing for query performance,
- and reproducibility via Docker.

This output was valuable because it linked database design decisions to data-engineering principles (e.g., constraint enforcement, surrogate vs. natural keys, data-type trade-offs). It captured both *technical correctness* and *academic justification*, reducing time spent rewriting explanatory text while improving cohesion between code and report. I retained the schema unchanged after cross-testing in MySQL, confirming that the generated explanations aligned with real implementation behavior.

Example of AI Failure or "Hallucination"

A clear failure occurred when I initially asked ChatGPT to infer *data-type lengths* and *foreign-key relationships* automatically from the CSVs. The AI generated an incomplete schema that:

- defined email columns as TEXT instead of VARCHAR(255),
- proposed a redundant user_id field not present in the source data, and
- ignored the composite relation between order_items and listings.

I recognized the issue during MySQL test loads. Foreign-key creation failed due to mismatched column names. The hallucination stemmed from AI assumptions rather than data inspection. I corrected it by explicitly validating column names from the cleaned CSVs and manually editing the schema before re-testing in Docker. This reinforced the need for human-driven data validation even when AI provides plausible-sounding suggestions.

Critical Reflection on Workflow and Human Oversight

Using AI significantly accelerated low-level drafting and ideation, especially for:

- report structuring (headings, flow, academic tone),
- summarizing design rationales concisely, and
- generating initial Python and SQL code templates.

However, deep reasoning still required human control:

- assessing *data quality anomalies* (e.g., detecting timestamp inconsistencies or column overflows) demanded manual CSV inspection;
- validating referential integrity and MySQL error handling required hands-on testing in Docker;
- reflecting on data-model trade-offs (e.g., normalization vs. denormalization) demanded conceptual judgment beyond what the AI could infer from syntax alone.

Ultimately, AI served as a productivity amplifier, it transformed my workflow from *code-first debugging* to *plan-then-verify*: I could iterate faster, but meaningful progress still depended on my own analytical verification, schema testing, and ability to discern hallucinations from valid insight.

In summary: GenAI tools streamlined documentation, brainstorming, and initial code generation, but human validation remained central for correctness, contextual reasoning, and critical reflection.